Python File Handling & Data Integration Assignment

Two questions, each worth 10 marks, covering integration of JSON/CSV and CSV/XML data sources using Python.

Question 1: Online Food Delivery Analysis (10 Marks)

A food delivery platform stores restaurant details in JSON format and customer order information in CSV format. Write a Python program to integrate both datasets, identify the restaurants receiving the highest number of orders, and generate a performance report.

Sample Input Data

restaurants.json (excerpt):

```
[
  {"restaurant_id": "R001", "name": "Spice Villa", "cuisine": "Indian", "location": "T. Nagar, Chennai",
"rating": 4.5},
  {"restaurant_id": "R002", "name": "Pizza Planet", "cuisine": "Italian", "location": "Anna Nagar, Chennai",
"rating": 4.2},
  {"restaurant_id": "R003", "name": "Dragon Wok", "cuisine": "Chinese", "location": "Velachery, Chennai",
"rating": 4.0},
  {"restaurant_id": "R004", "name": "Burger Barn", "cuisine": "American", "location": "Adyar, Chennai",
"rating": 3.8},
  {"restaurant_id": "R005", "name": "Sushi Central", "cuisine": "Japanese", "location": "OMR, Chennai",
"rating": 4.6},
  {"restaurant_id": "R006", "name": "Tandoori Nights", "cuisine": "Indian", "location": "Nungambakkam,
Chennai", "rating": 4.3}
]
  ...
```

orders.csv (excerpt):

```
order_id,restaurant_id,customer_name,order_amount,order_date
O1001,R001,Arun Kumar,450,2025-06-01
O1002,R002,Priya Sharma,620,2025-06-01
O1003,R001,Divya Menon,310,2025-06-02
O1004,R003,Karthik Raja,280,2025-06-02
O1005,R001,Sneha Reddy,540,2025-06-03
...
```

Python Program

```
"""
Online Food Delivery Analysis
------------------------------
Integrates restaurant details (JSON) with customer order data (CSV),
identifies the restaurants receiving the highest number of orders,
and generates a performance report.
"""

import json
import csv
from collections import defaultdict


def load_restaurants(json_path):
    """Load restaurant details from a JSON file into a dict keyed by restaurant_id."""
    with open(json_path, "r") as f:
        restaurants = json.load(f)
    return {r["restaurant_id"]: r for r in restaurants}
```

```
def load_orders(csv_path):
    """Load order records from a CSV file into a list of dicts."""
    with open(csv_path, "r", newline="") as f:
        reader = csv.DictReader(f)
        return list(reader)


def integrate_data(restaurants, orders):
    """Combine order data with restaurant details."""
    integrated = []
    for order in orders:
        rest = restaurants.get(order["restaurant_id"], {})
        integrated.append({
            "order_id": order["order_id"],
            "restaurant_id": order["restaurant_id"],
            "restaurant_name": rest.get("name", "Unknown"),
            "cuisine": rest.get("cuisine", "Unknown"),
            "location": rest.get("location", "Unknown"),
            "rating": rest.get("rating", "N/A"),
            "customer_name": order["customer_name"],
            "order_amount": float(order["order_amount"]),
            "order_date": order["order_date"],
        })
    return integrated


def analyze_performance(integrated_data):
    """Compute number of orders and total revenue per restaurant."""
    stats = defaultdict(lambda: {"orders": 0, "revenue": 0.0, "name": "",
                                 "cuisine": "", "location": "", "rating": ""})
    for rec in integrated_data:
        rid = rec["restaurant_id"]
        stats[rid]["orders"] += 1
        stats[rid]["revenue"] += rec["order_amount"]
        stats[rid]["name"] = rec["restaurant_name"]
        stats[rid]["cuisine"] = rec["cuisine"]
        stats[rid]["location"] = rec["location"]
        stats[rid]["rating"] = rec["rating"]
    return stats


def generate_report(stats):
    """Print a formatted performance report, ranked by number of orders."""
    ranked = sorted(stats.items(), key=lambda x: x[1]["orders"], reverse=True)

    print("=" * 80)
    print("RESTAURANT PERFORMANCE REPORT")
    print("=" * 80)
    print(f"{'Rank':<5}{'Restaurant':<18}{'Cuisine':<12}{'Orders':<8}"
          f"{'Revenue (₹)':<14}{'Avg Order (₹)':<14}{'Rating':<6}")
    print("-" * 80)

    for rank, (rid, s) in enumerate(ranked, start=1):
        avg_order = s["revenue"] / s["orders"] if s["orders"] else 0
        print(f"{rank:<5}{s['name']:<18}{s['cuisine']:<12}{s['orders']:<8}"
              f"{s['revenue']:<14.2f}{avg_order:<14.2f}{s['rating']:<6}")

    print("-" * 80)
    top_restaurant = ranked[0]
    print(f"\nTop performing restaurant: {top_restaurant[1]['name']} "
          f"with {top_restaurant[1]['orders']} orders "
          f"and ₹{top_restaurant[1]['revenue']:.2f} in revenue.")

    total_orders = sum(s["orders"] for _, s in ranked)
    total_revenue = sum(s["revenue"] for _, s in ranked)
    print(f"Total orders across all restaurants: {total_orders}")
    print(f"Total revenue across all restaurants: ₹{total_revenue:.2f}")
    print("=" * 80)


def main():
    restaurants = load_restaurants("restaurants.json")
    orders = load_orders("orders.csv")

    integrated_data = integrate_data(restaurants, orders)
    stats = analyze_performance(integrated_data)
```

```
    generate_report(stats)


if __name__ == "__main__":
    main()
```

Program Output

```
================================================================================
RESTAURANT PERFORMANCE REPORT
================================================================================
Rank Restaurant        Cuisine     Orders  Revenue (₹)   Avg Order (₹) Rating
--------------------------------------------------------------------------------
1    Spice Villa       Indian      7       3100.00       442.86        4.5
2    Pizza Planet      Italian     3       1360.00       453.33        4.2
3    Dragon Wok        Chinese     3       840.00        280.00        4.0
4    Sushi Central     Japanese    3       2170.00       723.33        4.6
5    Burger Barn       American    2       820.00        410.00        3.8
6    Tandoori Nights   Indian      2       760.00        380.00        4.3
--------------------------------------------------------------------------------

Top performing restaurant: Spice Villa with 7 orders and ₹3100.00 in revenue.
Total orders across all restaurants: 20
Total revenue across all restaurants: ₹9050.00
================================================================================
```

Question 2: Vehicle Insurance Management (10 Marks)

An insurance company stores vehicle information in CSV format and policy details in XML format. Write a Python program to combine the datasets, identify vehicles with expired insurance policies, and generate a renewal notification report.

Sample Input Data

vehicles.csv (excerpt):

```
vehicle_id,owner_name,vehicle_number,vehicle_type
V001,Arjun Mehta,TN01AB1234,Car
V002,Sneha Kapoor,TN02CD5678,Bike
V003,Ravi Shankar,TN03EF9012,Car
V004,Nisha Verma,TN04GH3456,Truck
V005,Manoj Tiwari,TN05IJ7890,Bike
...
```

policies.xml (excerpt):

```
<?xml version="1.0" encoding="UTF-8"?>
<policies>
    <policy>
        <vehicle_id>V001</vehicle_id>
        <policy_number>POL-3001</policy_number>
        <insurer>SafeDrive Insurance</insurer>
        <start_date>2025-01-15</start_date>
        <expiry_date>2026-01-14</expiry_date>
        <premium>8500</premium>
    </policy>
  ...
</policies>
```

Python Program

```
"""
Vehicle Insurance Management
----------------------------
Combines vehicle information (CSV) with policy details (XML),
identifies vehicles with expired insurance policies,
and generates a renewal notification report.
"""

import csv
import xml.etree.ElementTree as ET
from datetime import datetime, date


def load_vehicles(csv_path):
    """Load vehicle details from a CSV file into a dict keyed by vehicle_id."""
    with open(csv_path, "r", newline="") as f:
        reader = csv.DictReader(f)
        return {row["vehicle_id"]: row for row in reader}


def load_policies(xml_path):
    """Parse policy details from an XML file into a dict keyed by vehicle_id."""
    tree = ET.parse(xml_path)
    root = tree.getroot()
    policies = {}
    for policy in root.findall("policy"):
        vehicle_id = policy.find("vehicle_id").text
        policies[vehicle_id] = {
            "policy_number": policy.find("policy_number").text,
            "insurer": policy.find("insurer").text,
            "start_date": policy.find("start_date").text,
            "expiry_date": policy.find("expiry_date").text,
            "premium": float(policy.find("premium").text),
        }
```

```
    return policies


def combine_data(vehicles, policies):
    """Merge vehicle and policy records on vehicle_id."""
    combined = []
    for vid, vehicle in vehicles.items():
        policy = policies.get(vid, {})
        combined.append({
            "vehicle_id": vid,
            "owner_name": vehicle["owner_name"],
            "vehicle_number": vehicle["vehicle_number"],
            "vehicle_type": vehicle["vehicle_type"],
            "policy_number": policy.get("policy_number", "N/A"),
            "insurer": policy.get("insurer", "N/A"),
            "expiry_date": policy.get("expiry_date", None),
            "premium": policy.get("premium", 0),
        })
    return combined


def find_expired_policies(combined_data, reference_date=None):
    """Return records whose policy expiry_date is before the reference date."""
    ref = reference_date or date.today()
    expired = []
    for rec in combined_data:
        if rec["expiry_date"] is None:
            continue
        expiry = datetime.strptime(rec["expiry_date"], "%Y-%m-%d").date()
        if expiry < ref:
            rec["days_expired"] = (ref - expiry).days
            expired.append(rec)
    return sorted(expired, key=lambda r: r["days_expired"], reverse=True)


def generate_renewal_report(expired_records, reference_date=None):
    """Print a formatted renewal notification report."""
    ref = reference_date or date.today()
    print("=" * 100)
    print("VEHICLE INSURANCE RENEWAL NOTIFICATION REPORT")
    print(f"Report Date: {ref.strftime('%d-%b-%Y')}")
    print("=" * 100)

    if not expired_records:
        print("No vehicles with expired insurance policies. All policies are active.")
        print("=" * 100)
        return

    print(f"{'Vehicle No.':<14}{'Owner':<16}{'Type':<8}{'Policy No.':<12}"
          f"{'Insurer':<28}{'Expired On':<13}{'Days Ov.':<9}")
    print("-" * 100)

    for rec in expired_records:
        expiry_display = datetime.strptime(rec["expiry_date"], "%Y-%m-%d").strftime("%d-%b-%Y")
        print(f"{rec['vehicle_number']:<14}{rec['owner_name']:<16}{rec['vehicle_type']:<8}"
              f"{rec['policy_number']:<12}{rec['insurer']:<28}{expiry_display:<13}"
              f"{rec['days_expired']:<9}")

    print("-" * 100)
    print(f"\nTotal vehicles with expired policies: {len(expired_records)}")
    print("\nNotification Message:")
    for rec in expired_records:
        print(f"  -> Dear {rec['owner_name']}, your {rec['vehicle_type']} "
              f"({rec['vehicle_number']}) insurance policy {rec['policy_number']} "
              f"expired {rec['days_expired']} day(s) ago. Please renew immediately.")
    print("=" * 100)


def main():
    vehicles = load_vehicles("vehicles.csv")
    policies = load_policies("policies.xml")

    combined_data = combine_data(vehicles, policies)
    expired_records = find_expired_policies(combined_data)
    generate_renewal_report(expired_records)
```

```
if __name__ == "__main__":
    main()
```

Program Output

```
================================================================================================================
VEHICLE INSURANCE RENEWAL NOTIFICATION REPORT
Report Date: 22-Jul-2026
================================================================================================================
Vehicle No.   Owner           Type    Policy No.  Insurer                    Expired On   Days Ov.
----------------------------------------------------------------------------------------------------------------
TN03EF9012    Ravi Shankar    Car     POL-3003    SafeDrive Insurance        09-May-2025  439
TN01AB1234    Arjun Mehta     Car     POL-3001    SafeDrive Insurance        14-Jan-2026  189
TN05IJ7890    Manoj Tiwari    Bike    POL-3005    National General Insurance 19-Mar-2026  125
TN07MN3344    Suresh Pillai   Van     POL-3007    TruckSecure Insurance      14-Jun-2026  38
----------------------------------------------------------------------------------------------------------------

Total vehicles with expired policies: 4

Notification Message:
  -> Dear Ravi Shankar, your Car (TN03EF9012) insurance policy POL-3003 expired 439 day(s) ago. Please renew
immediately.
  -> Dear Arjun Mehta, your Car (TN01AB1234) insurance policy POL-3001 expired 189 day(s) ago. Please renew
immediately.
  -> Dear Manoj Tiwari, your Bike (TN05IJ7890) insurance policy POL-3005 expired 125 day(s) ago. Please renew
immediately.
  -> Dear Suresh Pillai, your Van (TN07MN3344) insurance policy POL-3007 expired 38 day(s) ago. Please renew
immediately.
================================================================================================================
```